

Planning in Artificial Intelligence

Giuseppe De Giacomo

Università degli Studi di Roma “La Sapienza”

Planning in AI

INTRODUCTION

AI Aims at Conceiving Systems that Act Autonomously

- Autonomy is one of the grand objectives of AI.
- Aims at building autonomous agents/robots that operate in changing, incompletely known, unpredictable environments.
- In other words: Aims at empowering the agent with the ability of
 - deliberating ...
 - how to act in the world ...
 - autonomously
- First person view: it is the agent/system that decides what to do



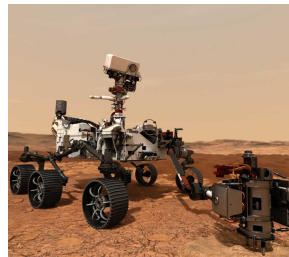
Examples of Autonomous Systems

Space Exploration

Delay in communication requires high-level of autonomy during the mission.

Planning and scheduling for temporal extended goals is a top research topic at NASA.

<https://mars.nasa.gov/mars2020/>



Interacting with Humans

Robots interacting with humans needs to properly schedule execution of complex actions that extends over time, as well as react to unexpected circumstances.



Autonomy is a Challenge in Business Process Management (BPM)

- BPM'21 Tutorial: Applications of Automated Planning for Business Process Management by Andrea Marrella and Tathagata Chakraborti



- BPM'21 Tutorial: RuM: Declarative Process Mining, Distilled by Anti Alman, Claudio Di Ciccio, Fabrizio Maria Maggi, Marco Montali and Han van der Aa



- BPM'21 Invited talk: Artificial Intelligence-based Declarative Process Synthesis for BPM by Giuseppe De Giacomo

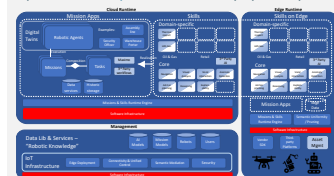


Autonomous Mobile Robots in Logistics

Complex multi-robot systems need highly synchronized behaviours to fulfil their job.

These robots need autonomously resolve unexpected clashes.

Sophisticated AMR platforms under study by industry



FinTech

- Smart Contracts Composition: systematic approach to compose many Smart Contracts (e.g. on Ethereum) from an extracted abstract specification to implement novel services for the user.

- Encoding Trading Strategies with Formal Methods:
 - Safety conditions: stop-loss, take-profit
 - Liveness conditions: order placement control
 - Interoperability with learning modules can forecast optimal order amounts
- Security & Fraud Detection: Process Mining to discover suspicious patterns of illegal activities e.g. money laundering
- Many AI + FinTech Academic Venues!
 - ICAPS's FinPlan workshop
 - ICAI's FinTech track



CHASE J.P.Morgan 2021 AI Faculty Award

Smart Manufacturing and Digital Twins

- Manufacturing as a service products to be manufactured are not known in advance and each product may differ from the products manufactured immediately before and immediately after it
- Analogies with Service Composition and Orchestration: synthesize the orchestrator
- Digital Twins platforms offer infrastructure to deploy orchestrations
- Automated exception handling is crucial



G. De Giacomo, R. Felli, B. Logan, F. Patrizi, S. Sardina: Situation Calculus for Controller Synthesis in Manufacturing Systems with First-Order State Representation. AIJ2021
G. De Giacomo, M. Favaria, F. Leotta, M. Maccario, L. Sile: Digital Twins Composition via Markov Decision Processes. ITBPM2021

KR: Have a Model of Environment and Use it to Act

Knowledge Representation:

“Equip agent with a model of the environment it acts in”

Reasoning about actions (classical view):

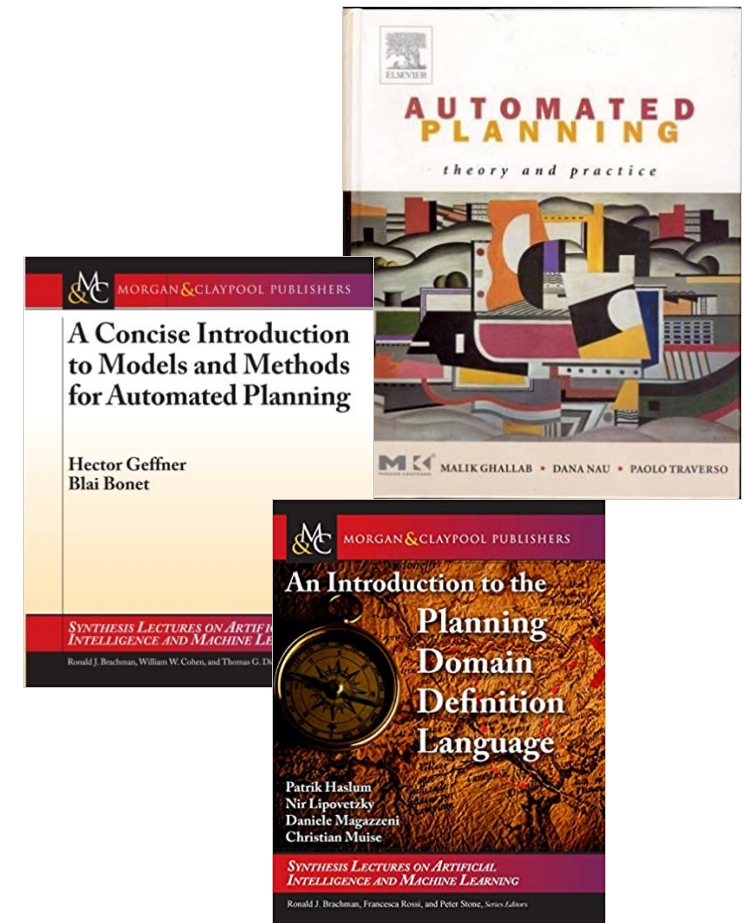
- Capture model of the environment with a **logical theory**
 - Preconditions for agent actions
 - Effects of agent actions + solution to “Frame Problem”
- **Multiple interpretations of the theory**
 - Multiple possible instantiations of the environment (one of them the **correct one**, but we do not know which)
- Reasoning (skeptical)
 - based on **logical implication**



Planning: Model the Environment as a Transition System

Planning:

- Inherit from KR the idea that environment can be represented in terms of
 - Preconditions for agent actions
 - Effects of agent actions + solution to “Frame Problem”
- But use them as a specification for generating a transition system
 - a single model vs a theory
- Reasoning
 - based on “model checking”
- Two notable cases:
 - Classical planning: everything determined by agent actions
 - Planning in nondeterministic domains (FOND):
 - Agent: instructs actions
 - Env: determine their effects



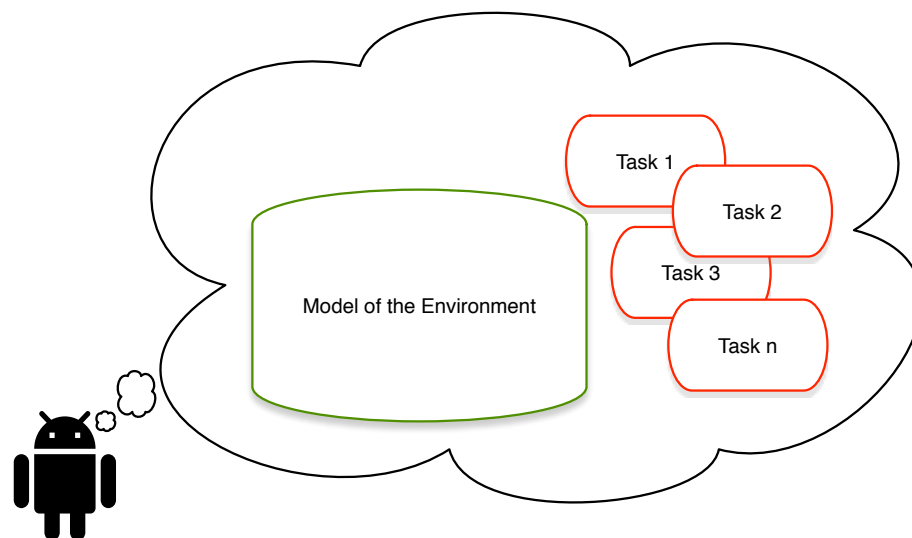
Focus on Terminating Tasks

Planning in AI:

- Is all about having a task specification or “goal” and producing a “plan” (or strategy or policy) to satisfy the task in the environment model.
- Which tasks?
 - A **task that terminates!**
 - Typically, just reaching a certain state in the environment

Why tasks that terminates?

- Because it is the agent that is planning/reasoning
- If the task would not terminate, the agent would be stuck into doing the same task forever
- But then, why bother with equipping it with a model of the environment and of the task at all?
- Note it is the agent, NOT the designer, who has such models

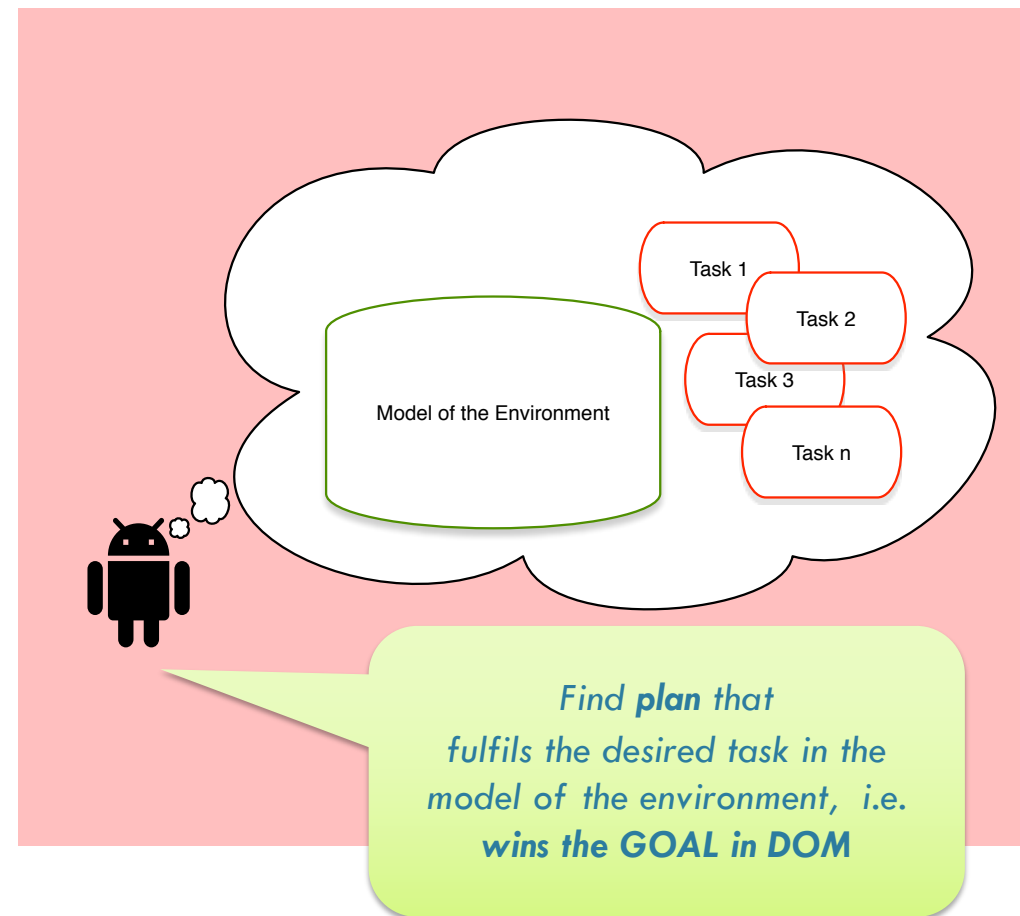


Planning in AI

PLANNING

Domains and Goals

- **Environment Model (DOM)**
 - Spec. of **environment's behaviors** of the world in response to **agent's action**
 - **Domain** expressed as with specific formalisms
 - STRIPS
 - ADL
 - PDDL
 - **Domain** is (or better generates) a **deterministic transition system**
- **Agent's Task (GOAL)**
 - Spec. of **agent's task**
 - **GOAL** expressed as **reaching a state of the domain with desired properties**
- **Find agent's plan/behavior/policy/strategy** that fulfills **GOAL** in **DOM**



Planning in Deterministic Domains

It's a transition system!

Deterministic domain (including initial state)

$\mathcal{D} = (2^{\mathcal{F}}, \mathcal{A}, s_0, \delta, \alpha)$ where:

- $\mathcal{F}$ **fluents** (atomic propositions)
- $\mathcal{A}$ **actions** (atomic symbols)
- $2^{\mathcal{F}}$ set of states
- s_0 initial state (initial assignment to fluents)
- $\alpha(s) \subseteq \mathcal{A}$ represents **action preconditions**
- $\delta(s, a) = s'$ with $a \in \alpha(s)$ represents **action effects (including frame)**.

Traces

A **trace** for $\mathcal{D}$ is a finite sequence:

$$s_0, a_1, s_1, \dots, a_n, s_n$$

where s_0 is the initial state, and $a_i \in \alpha(s_i)$ and $s_{i+1} = \delta(s_i, a_{i+1})$ for each i .

Goals, planning, and plans

Goal = propositional formula G on fluents

Planning = find a trace $s_0, a_1, s_1, \dots, a_n, s_n$ such that $s_n \models G$. (PSPACE-complete)

Plan = project traces on actions, i.e., return $a_1, \dots, a_n$.

Planning in Deterministic Domains

Winning states for deterministic domains

Let denote the **set of goal states** as:

$$\llbracket G \rrbracket = \{s \in \mathcal{S} \mid s \models G\}$$

and let's define the **(existential) preimage of a set $\mathcal{E}$** the following function:

$$PreE(\mathcal{E}) = \{s \in \mathcal{S} \mid \exists a \in \alpha(s). \delta(s, a) \in \mathcal{E}\}$$

Compute the set Win of winning states of the domain for goal G , i.e., states from which the agent can reach the goal G , by **least-fixpoint**:

- $Win_0 = \llbracket G \rrbracket$ (the goal states)
- $Win_{i+1} = Win_i \cup PreE(Win_i)$
- $Win = \bigcup_i Win_i$

```

Wold := ∅
W := ⌊G⌋
while (W ≠ Wold) {
    Wold := W
    W := W ∪ PreE(W)
}
return W
  
```

(Computing Win is linear in the number of states in $\mathcal{G}$)

Computing the winning strategy

Let's define $\omega : \mathcal{S} \rightarrow 2^{\mathcal{A}}$ as:

$$\omega(s) = \{a \in \alpha(s) \mid \text{if } s \in Win_{i+1} - Win_i \text{ then } \delta(s, a) \in Win_i\}$$

Every way of restricting $\omega(s)$ to return only one action (chosen arbitrarily) gives a **winning strategy** to reach G , i.e., a **plan**.

Planning in Deterministic Domains

- The kind of plan just computed is called “**Universal Plan**”
 - It is **able to say what to do to win in every single state!**
 - **If** for any reason during the execution of the plan the agent find itself in an **unexpected state**
 - i.e.: the agent expected an effect but s/he got another one

Then the **universal plan knows what to do** to win anyway
- Typically in planning it is sufficient to have a **classical plan** (not a universal one), i.e., a **sequence of actions that from the initial state leads the agent to a state satisfying the goal.**
- To compute **classical plans** there are **better algorithms** (in fact exceptionally fast) - see later in the course.

Planning in AI

STRIPS

STRIPS (Stanford Research Institute Problem Solver)

- **STRIPS** [Fikes and Nilsson (1971)] was originally developed for the pioneering autonomous robot **Shakey**
- STRIPS specifies a **transition system** through a simple use of **logic**
 - **Fluents**
 - **propositional atomic formulas**
 - **Actions** specification:
 - **Preconditions**: just a conjunction of (propositional) fluents
 - **Effects**: a conjunction of positive and negative fluents
 - **Frame**: every fluent not mentioned in the effects remains unchanged
 - **Initial situation** specification
 - A conjunction of fluents (all other fluents are false)
 - **Goal** specification
 - A conjunction of fluents (all other fluents can assume any value)
- STRIPS is **simple** but very **much used** in planning

STRIPS

- A problem in STRIPS is a tuple $P = \langle F, A, I, G \rangle$:
 - F stands for set of all fluents (Boolean vars)
 - A stands for set of all actions (also called operators)
 - $I \subseteq F$ stands for initial situation
 - $G \subseteq F$ stands for goal condition
- Actions $a \in A$ represented by
 - the **Precondition list** $\text{Pre}(a) \subseteq F$ (the fluents that need to be true to execute the action)
 - the **Add list** $\text{Add}(a) \subseteq F$ (the fluents that will become positive)
 - the **Delete list** $\text{Del}(a) \subseteq F$ (the fluents that will become negative)
(the fluents not in $\text{Add}(a)$ and $\text{Del}(a)$ remain unchanged, solving the frame problem)

STRIPS

A STRIPS problem $P = \langle F, A, I, G \rangle$ determines transition system $D = (S, A, \text{delta}, \text{alpha})$ and a goal states $[[G]]$ where:

- The **states** $s \in S$ are collections of fluents from F (valuations over F)
- The **initial state** s_0 is I (initially a fluent in F is true iff it's in I)
- The **possible actions** $\text{alpha}(s)$ are the actions $a \in A$ such that $\text{Pre}(a) \subseteq s$

$$\text{alpha}(s) = \{a \in A \mid \text{Pre}(a) \subseteq s\}$$

- The **next state** is

$$s' = s \setminus \text{Del}(a) \cup \text{Add}(a)$$

- The **goal states** are such that

$$[[G]] = \{s \in S \mid G \subseteq s\}$$

Planning in AI

ADL

ADL (Action Description Language)

- **ADL** [Pednault (1986)] was originally developed as an improved version of STRIPS, also influenced by Situation Calculus, contains important ideas incorporated in Reiter's Situation Calculus Preconditions Axioms and Successor State Axioms
- ADL specifies a **transition system** through a more sophisticated use of **logic**
 - **Fluents**
 - **propositional atomic formulas, or FOL atomic formulas to be instantiated on a finite domain!**
 - **Actions** specification:
 - **Preconditions:** **arbitrary propositional/FOL formula on fluents**
 - **Effects:** a conjunction of positive and negative fluents, but can have **arbitrary propositional/FOL conditions;** effects are **conditional:** when performing action a
 - if **condition** $\text{Cond}_i(a)$ holds now
 - Then **effect** $\text{Add}_i(a), \text{Del}_i(a)$ hold next
$$\text{Cond}_i(a) \sim > \text{Add}_i(a), \text{Del}_i(a) \quad (i=1, \dots, n)$$
 - **Frame:** every fluent not mentioned in the effects remains unchanged
 - **Initial situation** specification
 - A conjunction of fluents (all other fluents are false)
 - **Goal** specification
 - An **arbitrary propositional/FOL formula on fluents**

ADL

An **ADL** problem $P = \langle F, A, I, G \rangle$ determines transition system $D = (S, A, \delta, \alpha)$ and a goal states $[[G]]$, *after having instantiated all FOL formulas (grounding) and obtained a corresponding propositional specification of P* , where :

- The **states** $s \in S$ are collections of fluents from F (valuations over F)
- The **initial state** s_0 is I (initially a fluent in F is true iff it's in I)
- The **possible actions** $\alpha(s)$ are the actions $a \in A$ such that $\text{Pre}(a) \subseteq s$

$$\alpha(s) = \{a \in A \mid s \models \text{Pre}(a)\}$$

- The **next state** is

$$s' = s \setminus (\bigcup_i \text{Del}_i(a) \text{ where } s \models \text{Cond}_i(a)) \cup \bigcup_i \text{Add}_i(a) \text{ where } s \models \text{Cond}_i(a)$$

- The **goal states** are such that

$$[[G]] = \{s \in S \mid s \models G\}$$

ADL

Theorem: Any ADL (and hence STRIPS) domain specification can be (polynomially) expressed into a Reiter's Situation Calculus Basic Action Theory.

Theorem: Any Reiter's Situation Calculus Basic Action Theory, (1) on finite object domain (2) with initial situation description being a DB, can be (polynomially) expressed into ADL (but not in STRIPS).

Theorem: ADL Planning tools can be used for solving planning in Reiter's Situation Calculus Basic Action Theory on finite domain with initial situation description being a DB.

Planning in AI

PDDL

Example STRIPS

Axiomatize the following scenario, appropriately with action precondition and effect axioms, and obtain successor state axioms.

Fluents:

- **boxClosed(s)** - The box is closed in situation s.
- **robotCloseToBox(s)** - The robot is close to the box in situation s.
- **objectInsideBox(s)** - The object is inside the box in situation s.

*Task: extract the object from the box
i.e.*

Goal: not objectInsideBox

Actions:

- **moveCloseToBox** - The robot moves close to the box. This can be done if the robot is not close to the box, and has the effect that the robot will be close to the box.
- **openBox** - The robot opens the box. This can be done if the robot is close to the box and the box is closed, and has the effect that the box will be open (that is, not closed).
- **extractObject** - The robot extracts the object inside the box. This requires that the box is open and the robot is close to the box, and has the effect that the object will be outside the box.

Initial situation description: Initially the robot is not close to the box, the box is closed, and the object is inside the box.

Example STRIPS

```
(define (domain box)                                domain
(:requirements :strips :negative-preconditions)
(:predicates (boxClosed)
              (robotCloseToBox)
              (objInsideBox)
)

(:action moveCloseToBox
  :parameters ()
  :precondition (and (not (robotCloseToBox)))
  :effect (and (robotCloseToBox))
)

(:action openBox
  :parameters ()
  :precondition (and (robotCloseToBox) (boxClosed))
  :effect (and (not (boxClosed)))
)

(:action extractObject
  :parameters ()
  :precondition (and (not (boxClosed)) (robotCloseToBox))
  :effect (and (not (objInsideBox)))
))
```

```
(define (problem box-1)                             problem
(:domain box)
(:objects )

(:init
  ;; (not (robotCloseToBox))
  (boxClosed)
  (objInsideBox)
)

(:goal
  (not (objInsideBox))
))
```

Example ADL (Adapted from Situation Calculus)

We consider 3 action each with its preconditions and effects (sort of postconditions):

- $drive(t, \ell_1, \ell_2)$
 - ▶ **Pre:** $at(t, \ell_1, s) \wedge connected(\ell_1, \ell_2)$
 - ▶ **Effects:** $at(t, \ell_2, s') \wedge \neg at(t, \ell_1, s')$ where $s' = do(drive(t, \ell_1, \ell_2), s)$
- $load(p, t)$
 - ▶ **Pre:** $\exists \ell. at(p, \ell, s) \wedge at(t, \ell, s) \wedge \neg \exists t'. on(p, t', s)$
 - ▶ **Effects:** $\forall \ell. \neg at(p, \ell, s') \wedge on(p, t, s')$ where $s' = do(load(p, t), s)$
- $unload(t)$
 - ▶ **Pre:** $\exists p. on(p, t, s)$
 - ▶ **Effects:** $(\forall p, \ell. on(p, t, s) \wedge at(t, \ell, s) \rightarrow at(p, \ell, s')) \wedge$
 $(\forall p. on(p, t, s) \rightarrow \neg on(p, t, s'))$
NB: equivalent to $\forall p. on(p, t, s) \rightarrow ((\forall \ell. at(t, \ell, s) \rightarrow at(p, \ell, s')) \wedge \neg on(p, t, s'))$
 where $s' = do(unload(t), s)$

Example

```
(define (domain logistics)
```

```
(:requirements :adl)
```

```
(:types
```

```
  pack vehicle - object
```

```
  truck - vehicle
```

```
  loc
```

```
)
```

```
(:predicates
```

```
  (connected ?l1 ?l2 - loc) ;; nonfluent
```

```
  (at ?p - object ?l - loc) ;; fluent
```

```
  (on ?p - pack ?t - truck) ;; fluent
```

```
)
```

domain

```
(:action load
```

```
  :parameters (?p - pack ?t - truck)
```

```
  :precondition (exists (?l - loc) (and (at ?t ?l) (at ?p ?l)))
```

```
  :effect (and
```

```
    (on ?p ?t)
```

```
    (forall (?l - loc) (not (at ?p ?l))))
```

```
)
```

```
)
```

```
(:action drive
```

```
  :parameters (?t - truck ?from ?to - loc)
```

```
  :precondition (and (at ?t ?from) (connected ?from ?to))
```

```
  :effect (and (at ?t ?to) (not (at ?t ?from))))
```

```
)
```

```
(:action unload
```

```
  :parameters (?t - truck)
```

```
  :precondition (exists (?p - pack) (on ?p ?t))
```

```
  :effect (and
```

```
    (forall(?l - loc ?p - pack) (when (and (on ?p ?t) (at ?t ?l)) (at ?p ?l))))
```

```
    (forall(?p - pack) (when (on ?p ?t) (not (on ?p ?t)))))
```

```
)
```

```
)
```

Example



```
(define (problem logistics-1)
  (:domain logistics)
  (:objects
    truck1 - truck
    package1 - pack
    package2 - pack
    package3 - pack
    location1 - loc
    location2 - loc
    location3 - loc
    location4 - loc
  )
```

```
(:init
  (at truck1 location1)
  (at package1 location1)
  (at package2 location1)
  (at package3 location1)

  ;; no atoms involving on

  (connected location1 location2)
  (connected location2 location3)
  (connected location3 location4)
  (connected location4 location1)
  (connected location1 location4)
  (connected location2 location4)
)
```

```
(:goal (and
  (at package1 location4)
  (at package2 location4)
  (at package3 location4))
)
```

problem

Planning in AI

PLANNING IN NONDETERMINISTIC DOMAINS

Example

Consider the following variant of the Yale Shooting Scenario, which has to do with a shooting a turkey named Fred, with a gun that can be defective. Find a plan to kill the turkey.

Yale Shooting Scenario variant:

- **Fluents:**
 - **alive** - The turkey is alive in the current situation.
 - **loaded**– The gun is loaded in the current situation.
 - **jammed**– The gun is jammed in the current situation.
- **Actions:**
 - **load** – Load the gun. Requires that the gun is not loaded; has two alternative possible effects:
 - The gun is loaded and is not jammed
 - The gun is loaded and is jammed
 - **shoot** - Shoot the gun. Requires that the gun be loaded; has a conditional effect:
 - If the gun is not jammed, then it unloads the gun and kills the turkey (not alive)
 - If the gun is jammed, then it unjams it
 - **wait** – A no-op; Can always be performed and has no effect on any fluent
- **Initial situation description:**
 - Initially the turkey is alive, the gun is not loaded, and it is not jammed.
- **Goal:**
 - Kill the turkey, i.e., turkey is not alive

Use PDDL “oneof” for expressing alternative effects.

Example

- **Fluents** (propositional):
alive, loaded, jammed
- **Actions:**
 - load
 - Pre:** not loaded
 - Effects:**
 - oneof**
 - loaded and not jammed
 - loaded and jammed
 - shoot
 - Pre:** loaded
 - Effects:**
 - if* not jammed *then* not loaded and not alive
 - if* jammed *then* not jammed
 - wait
 - Pre:** *none*
 - Effects:** *none*
- **Initial situation description:**
alive and not loaded and not jammed
- **Goal:**
not alive

Example

```
(define (domain yale-shooting-nondeterministic)
  (:requirements :adl :non-deterministic)
```

```
  (:predicates
    (alive)    ;; fluent
    (loaded)   ;; fluent
    (jammed)   ;; fluent
  )
```

```
(define (problem yale-1)
  (:domain box yale-shooting-nondeterministic)
  (:objects )
```

```
  (:init
    (alive)
    ;;(not loaded))
    ;;(not jammed))
  )
```

```
  (:goal
    (not (alive))
  ))
```

problem

```
(:action load
  :parameters ()
  :precondition (not (loaded))
  :effect (oneof
    (and (loaded) (jammed))
    (and (loaded) (not (jammed)))
  )
)
```

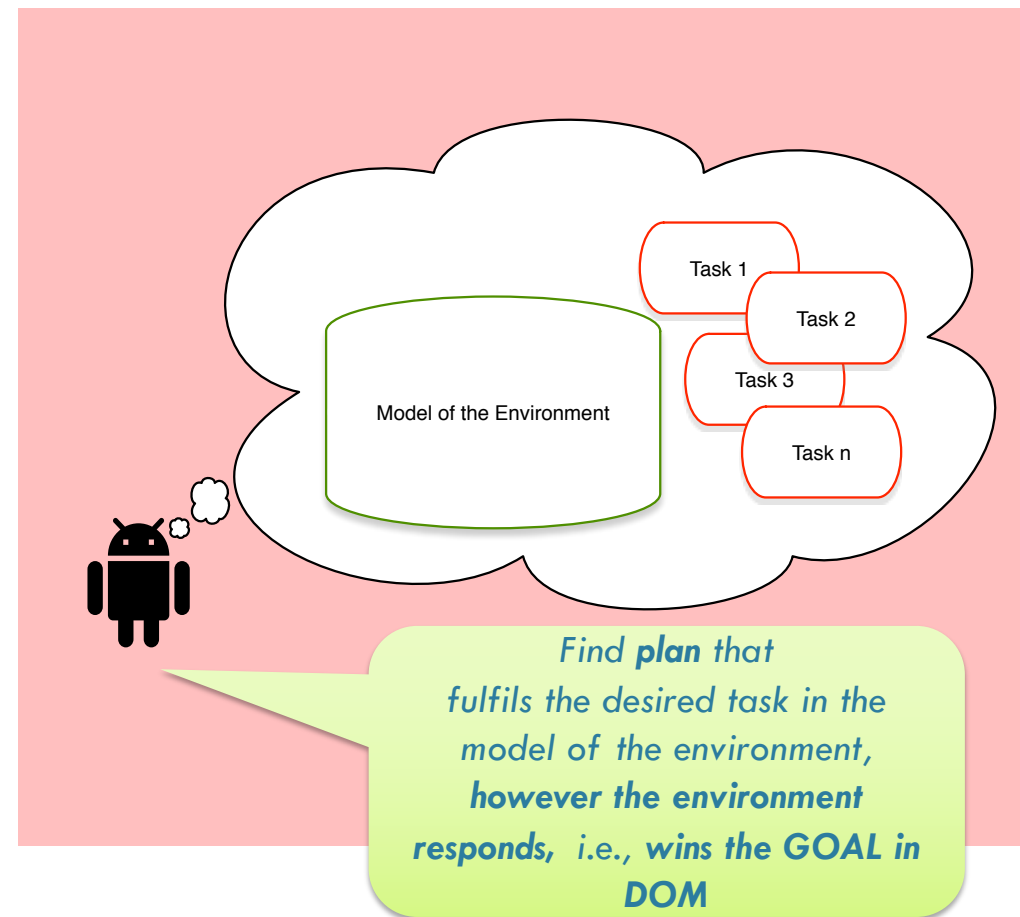
domain

```
(:action shoot
  :parameters ()
  :precondition (loaded)
  :effect (and
    (when (not (jammed)) (and (not loaded) (not alive)))
    (when (jammed) (not (jammed)))
  )
)
```

```
(:action wait
  :parameters ()
  :precondition ()
  :effect ()
)
```

Planning in Nondeterministic Domains

- **Environment Model (DOM)**
 - Spec. of **environment's behaviors** of the world in response to **agent's action**
 - **Domain** expressed as with specific formalisms
 - PDDL
 - **Domain** is (or better generates) a **non-deterministic transition system** (i.e., a **game arena for two players agent and env!**)
- **Agent's Task (GOAL)**
 - Spec. of **agent's task**
 - **GOAL** expressed as **reaching a state of the domain with desired properties**
- **Find agent's plan**/behavior/policy/strategy that fulfills **GOAL** in **DOM**



Planning in Nondeterministic Domains

Nondeterministic domain (including initial state)

$\mathcal{D} = (2^{\mathcal{F}}, \mathcal{A}, s_0, \delta, \alpha)$ where:

- $\mathcal{F}$ **fluents** (atomic propositions)
- $\mathcal{A}$ **actions** (atomic symbols)
- $2^{\mathcal{F}}$ set of states
- s_0 initial state (initial assignment to fluents)
- $\alpha(s) \subseteq \mathcal{A}$ represents **action preconditions**
- $\delta(s, a, s')$ with $a \in \alpha(s)$ represents **action effects (including frame)**.

*It's a **game arena** between agent controlling actions and env controlling fluents in the domain!*

Who controls what?

Fluents controlled by **environment**

Actions controlled by **agent**

Observe: $\delta(s, a, s')$

Goals, planning, and plans

Goal = propositional formula G on fluents

Planning = **game** between two players:

agent tries to force eventually reaching G no matter how other **environment** behave.

Plan = **strategy** to **win** the game.

*Fully Observable Nondeterministic Domains
Strong Planning, i.e., considers env adversarial*

(FOND_{sp} is EXPTIME-complete)

Planning in Nondeterministic Domains

Winning states for nondeterministic domains

Let denote the **set of goal states** as:

$$\llbracket G \rrbracket = \{s \in \mathcal{S} \mid s \models G\}$$

and let's define the **(adversarial preimage of a set $\mathcal{E}$)** the following function:

$$PreAdv(\mathcal{E}) = \{s \in \mathcal{S} \mid \exists a \in \alpha(s). \forall s' \in \mathcal{S}. \delta(s, a, s') \supset s' \in \mathcal{E}\}$$

Compute the set Win of winning states of the domain for goal G , i.e., states from which the agent can reach the goal G , by **least-fixpoint**:

- $Win_0 = \llbracket G \rrbracket$ (the goal states)
- $Win_{i+1} = Win_i \cup PreAdv(Win_i)$
- $Win = \bigcup_i Win_i$

```

Wold := ∅
W := ⌊G⌋
while (W ≠ Wold) {
    Wold := W
    W := W ∪ PreAdv(W)
}
return W

```

(Computing Win is linear in the number of states in $\mathcal{G}$)

Computing the winning strategy

Let's define $\omega : \mathcal{S} \rightarrow 2^{\mathcal{A}}$ as:

$$\omega(s) = \{a \in \alpha(s) \mid \text{if } s \in Win_{i+1} - Win_i \text{ then } \forall s'. \delta(s, a, s') \supset s' \in Win_i\}$$

Every way of restricting $\omega(s)$ to return only one action (chosen arbitrarily) gives a **winning strategy** to reach G , i.e., a **plan**.

Planning in AI

PLANNING IN NONDETERMINISTIC DOMAINS WITH STOCHASTIC FAIRNESS

Example

Consider the following another variant of the Yale Shooting Scenario, which has to do with a shooting a turkey named Fred, with a gun that can be defective. Find a plan to kill the turkey.

Yale Shooting Scenario variant:

- **Fluents:**
 - **alive** - The turkey is alive in the current situation.
 - **loaded**– The gun is loaded in the current situation.
- **Actions:**
 - **load** – Load the gun. Requires that the gun is not loaded; has two alternative possible effects:
 - The gun is loaded
 - nothing
 - **shoot** - Shoot the gun. Requires that the gun be loaded; has the following effect:
 - it unload the gun and kills the turkey (not alive)
 - **wait** – A no-op; Can always be performed and has no effect on any fluent
- **Initial situation description:**
 - Initially the turkey is alive, and the gun is not loaded.
- **Goal:**
 - Kill the turkey, i.e., turkey is not alive

*Use PDDL “**oneof**” for expressing alternative effects.*

Example

- **Fluents** (propositional):
alive, loaded

- **Actions:**

load

Pre: not loaded

Effects:

oneof

loaded

not loaded

shoot

Pre: loaded

Effects:

loaded and not alive

wait

Pre: *none*

Effects: *none*

- **Initial situation description:**

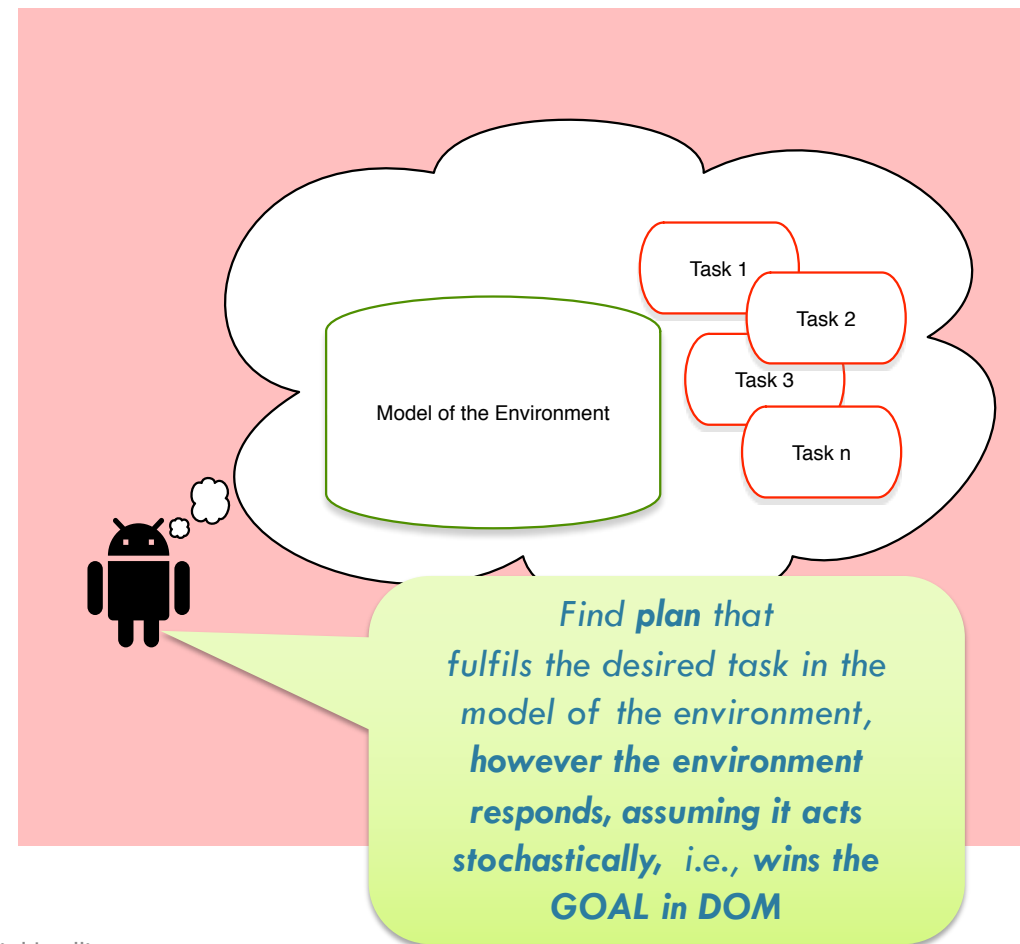
alive and not loaded

- **Goal:**

not alive

Planning in Nondeterministic Domains with Stochastic Fairness

- **Environment Model (DOM)**
 - Spec. of **environment's behaviors** of the world in response to **agent's action**
 - **Domain** expressed as with specific formalisms
 - PDDL
 - **Domain** is (or better generates) a **non-deterministic transition system** (i.e., a **game arena for two players agent and env!**)
 - Env acts **stochastically** and **not adversarially**
- **Agent's Task (GOAL)**
 - Spec. of **agent's task**
 - **GOAL** expressed as **reaching a state of the domain with desired properties**
- **Find agent's plan/behavior/policy/strategy that fulfills GOAL in DOM**



Planning in Nondeterministic Domains with Stochastic Fairness



SAPIENZA
UNIVERSITÀ DI ROMA

Nondeterministic domain (including initial state)

$\mathcal{D} = (2^{\mathcal{F}}, \mathcal{A}, s_0, \delta, \alpha)$ where:

- $\mathcal{F}$ **fluents** (atomic propositions)
- $\mathcal{A}$ **actions** (atomic symbols)
- $2^{\mathcal{F}}$ set of states
- s_0 initial state (initial assignment to fluents)
- $\alpha(s) \subseteq \mathcal{A}$ represents **action preconditions**
- $\delta(s, a, s')$ with $a \in \alpha(s)$ represents **action effects (including frame)**.

*It's a **game arena** between agent controlling actions and env controlling fluents in the domain!*

Who controls what?

Fluents controlled by **environment**, though under **stochastic fairness**:

(i.e., all effects will happen with non-zero (unknown) probability)

Actions controlled by **agent**

Observe: $\delta(s, a, s')$

Goals, planning, and plans

Goal = propositional formula G on fluents

Planning = **agent**, in spite of the **environment**, stays in an area from where is possible to reach G

*(with the cooperation of **environment**! it is not a pure adversarial game!)*

Plan = **strategy** to stay within the good area.

*Fully Observable Nondeterministic Domains
Strong Cyclic Planning, i.e., considers env
acting stochastically*

(FOND_{sc} is EXPTIME-complete)

Existential and universal preimages wrt the environment

$$\begin{aligned} PreE(a, \mathcal{E}) &= \{s \in S \mid a \in \alpha(s). \exists s' \in S. \varrho(s, a, s') \wedge s' \in \mathcal{E}\} \\ PreA(a, \mathcal{E}) &= \{s \in S \mid a \in \alpha(s). \forall s' \in S. \varrho(s, a, s') \supset s' \in \mathcal{E}\} \end{aligned}$$

Agent forces that always agent reach a final state if environment cooperates

The winning condition of the game is defined by two nested fixpoints, a **greatest-fixpoint** (for safety) and **least fixpoint** (for reachability):

$$Safe = \nu X. \mu Y. \llbracket G \rrbracket \cup \bigcup_{a \in \mathcal{A}} (PreA(a, X) \cap PreE(a, Y))$$

This gives rise to the following nested fixpoint computation:

- $X_0 = \mathcal{S}$ (all states of the domain $\mathcal{D}$)
- $X_{i+1} = Y_{i+1} = \mu Y. \llbracket G \rrbracket \cup \bigcup_{a \in \mathcal{A}} (PreA(a, X_i) \cap PreE(a, Y))$
- $Safe = \bigcap_i X_i$

where $\mu Y. \llbracket G \rrbracket \cup \bigcup_{a \in \mathcal{A}} (PreA(a, X_i) \cap PreE(a, Y))$ is computed as

- $Y_{i,0} = \llbracket G \rrbracket$ (the final states of $\mathcal{G}$)
- $Y_{i,j+1} = Y_{i,j} \cup \bigcup_{a \in \mathcal{A}} (PreA(a, X_i) \cap PreE(a, Y_{i,j}))$
- $Y_i = \bigcup_j Y_{i,j}$

(Computing each Y_i is linear in the number of states in $\mathcal{D}$, hence computing $Safe$ is quadratic.)

Computing the winning strategy

We can stratify *Safe* according to when a state enters the least fixpoint:

- $Reach_1 = \llbracket G \rrbracket$,
- $Reach_{j+1} = Reach_j \cup PreA(a, Safe) \cap PreE(a, Reach_j)$.

Note that $Safe = \bigcup_{j \leq |S|} Reach_j$.

Let's define $\omega : S \rightarrow 2^{\mathcal{A}}$ as:

$$\omega(s) = \{a \in \alpha(s) \mid \text{if } s \in Reach_{j+1} - Reach_j \text{ then } \exists s' \in S. \varrho(s, a, s') \wedge s' \in Reach_j\}$$

Every way of restricting $\omega(s)$ to return only one action (chosen arbitrarily) gives a **winning strategy**, i.e., a **plan**, to eventually reach G (with the help of the stochasticity of the environment).